# From Spare Compute to Cumulative Mathematics

*An open-source strategy for agent-assisted research on hard problems*

Will Cook*

September 2026

Abstract

We want someone with a mathematical idea, spare compute, or time to review a proof to be able to contribute to an ongoing research problem without first building a research environment. Plectis puts the papers, formal source, experiments, and tools in a public Git repository. A contributor can clone it, work on one question with an agent or by hand, and return a result for review. Accepted contributions retain their evidence and credit.

The repository currently covers eight open Erdős problems. A useful return may be a proof, but it may also be a counterexample, a corrected reference, an explanation of why a method fails, or a repair to the tools. The proposal is that these contributions should accumulate together: later attempts should have both better mathematics to draw on and a better environment for doing the work. This paper describes the contribution process, its precedents, and the tests needed to assess it. The public workflow is implemented, but its usefulness to outside contributors and its effect on research productivity have not yet been established. No result reported in this paper comes from an outside contributor; the evidence behind it is an implemented workflow, internal agent rehearsals operated by the author, and deterministic checks of the repository's own contracts.

## 1 The strategy

The project began with one undergraduate, without an institutional research team or a dedicated compute allocation. Preparing an agent to do useful work required more than choosing a model. It required locating the relevant mathematics, setting up proof checks, keeping track of unsuccessful attempts, and writing an account that could be understood after the conversation ended. The reason for publishing the environment is to let others use and improve that work.

A contributor need not take on a whole problem. A mathematician may suggest a missing lemma or find a counterexample. A programmer may run an exact calculation. A Lean contributor may formalise an argument or discover that its formal statement is wrong. Reviewers and expositors may supply what a long agent run could not: a sound judgement of the result and an explanation of its main idea. Contributions to the software are welcome on the same terms.

The eight Erdős problems give this work a mathematical purpose. A run can leave one of them open and still establish something worth preserving. Tao's discussion of learning from the study of a problem is relevant here [3]: a failed construction may reveal an invariant, expose a limitation of a technique, or suggest a connection elsewhere. Such a result needs an account of why the approach seemed plausible and where it stopped. His Navier–Stokes example raises the converse concern: publishing a final construction while concealing its development may lose much of the insight that led to it [4].

We therefore publish selected research records while the problems are still open. The records include proved statements, computations, references, counterexamples, and unresolved steps. An agent retrieves the relevant work, saves its attempted proofs and check results, and updates the affected files when a result is accepted. The systems paper's composite-dilation example follows an unsuccessful extension through this process.

*Authorship and AI use.* Will Cook built and directed the research infrastructure. AI agents produced the mathematical interpretations, proofs, prose, and software. Cook maintains the project and is responsible for its presentation; this is not a claim of personal mathematical verification or independent specialist review.

### Research and review

The public workflow has two recurring jobs. The first investigates a mathematical question. The second checks how the result fits the existing work and updates the paper and repository. The skill files call these *discovery* and *stewardship*. One agent can alternate between them; contributors can also divide the work.

The second job is needed because a successful proof attempt does not settle how the result should be presented or used. Five new declarations may amount to one routine lemma. A short counterexample may eliminate a line of research that was consuming most of the compute. A literature search may uncover a stronger theorem than the one leading the paper. The review changes the account of the result and, when appropriate, the next question to investigate.

The `coupled-research skill` passes source revisions, results, and check records between these jobs. A new theorem, correction, or review can trigger another pass. When nothing has changed, the jobs yield. The systems paper explains the implementation; the navigation paper examines how a new agent finds the mathematics and its checks. All of these public jobs can run without the private workbench.

### What one clone lets a contributor do

A newcomer can ask an agent to explain the repository before choosing a task. The `explanation skill` directs it to the public papers and source, with the level of detail adapted to the reader. Figure 1 gives examples of work that can follow. A contributor can also read a problem paper directly and proceed without an agent.

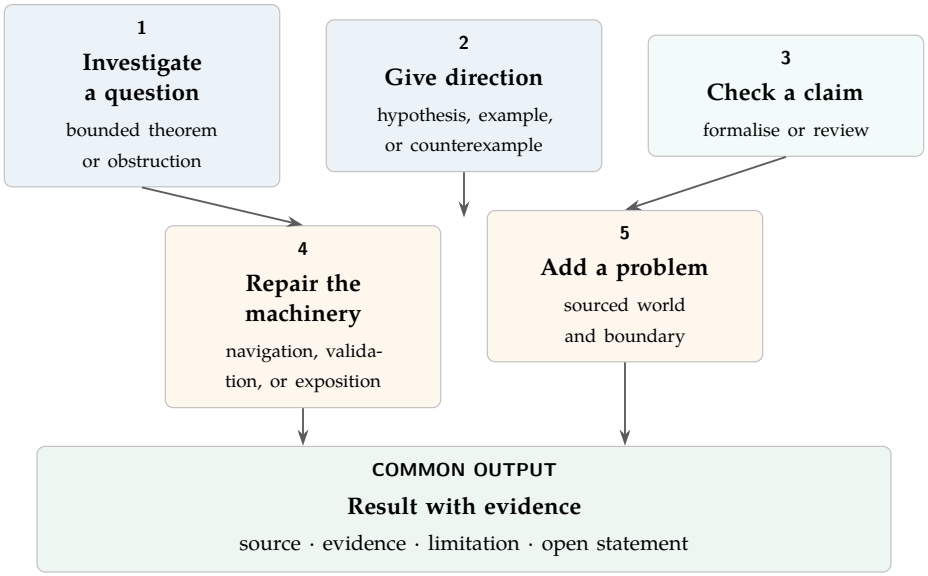


**Figure 1:** Ways to contribute, from investigating a question to improving the tools.

## 2 Where further work is needed

Research throughput depends on more than the number of agents running. Mathematical direction determines which questions they attempt; the available literature and tools determine what they can build on; verification and review determine which outputs become usable results. These constraints suggest two ways to contribute. A theorem, counterexample, or correction changes the mathematical record. Better retrieval, validation, or exposition changes the cost of using that record in later work.

For example, an unsuccessful extension may reveal a missing hypothesis. Recording that obstruction changes the next mathematical attempt. If the attempt also exposed a stale source link, repairing the link improves the environment for every subsequent reader. The two changes have different checks and can

be adopted and credited separately. Figure 2 shows how contributions enter this cycle. Its effect on productivity is an empirical question for Section 12.

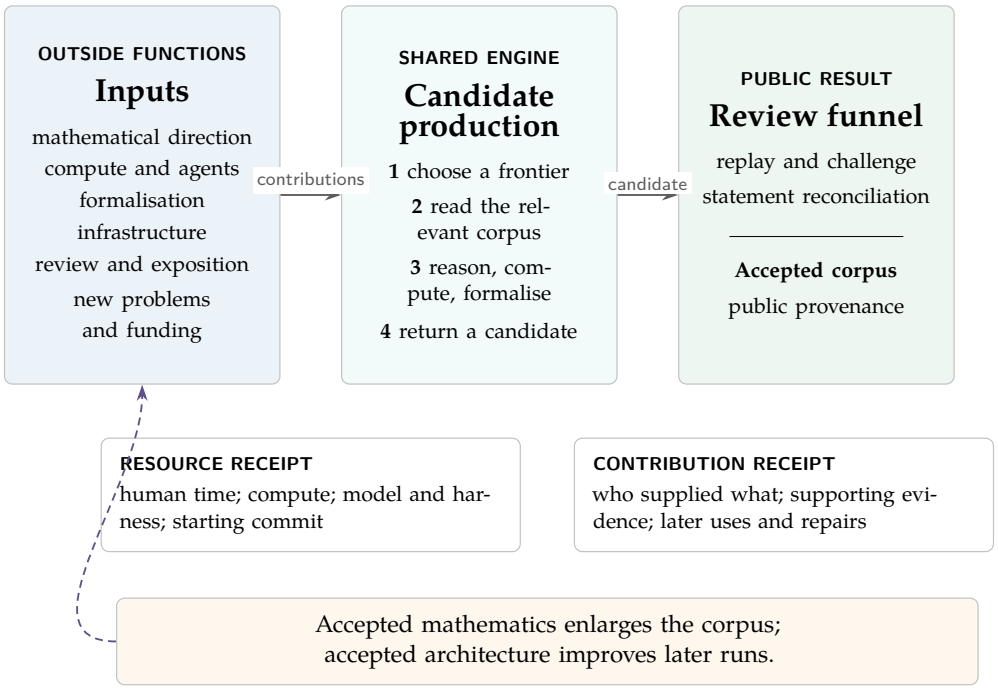


**Figure 2:** Contributors supply ideas, computation, formalisation, software, and review. The accepted work updates the mathematics or the tools used in later attempts.

A versioned repository also permits longitudinal evaluation. A fixed, versioned problem world can be revisited as models, runners, and compute budgets change. Comparisons must disclose the starting commit, model, scaffolding, number of attempts, compute, and review procedure. Otherwise a success says little about which variable changed. A public failure record is equally important: capability claims are badly distorted when successes are announced and the number and cost of failed attempts are hidden [2].

# 3 Why begin with hard mathematics

Mathematical work can travel with the tools needed to examine it. A clone can contain papers, exact computations, and formal proofs, and a contributor can check one lemma without first completing the entire argument. This makes mathematics a practical starting point for the proposal.

Computation helps choose what to try. It can find a counterexample, compare two formulations, or reveal a pattern that suggests a proof. The record needs the inputs, code, output, and interpretation, so a later reader can distinguish what was calculated from what was conjectured.

The eight selected problems provide different kinds of work (Figure 3). Their value as research environments will depend on the questions and results they produce, rather than on their recognisable problem numbers.

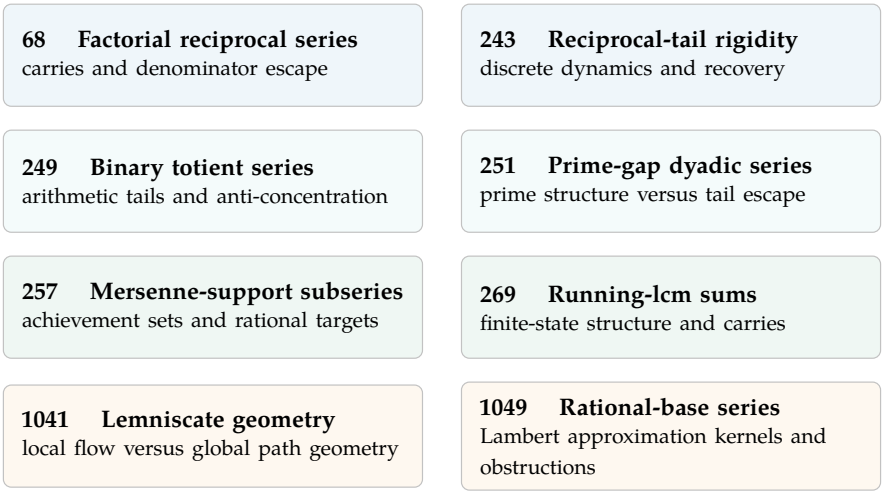


**Figure 3:** Eight distinct research environments. The results map and the problem papers state their exact frontiers; this strategy paper does not reproduce the specialist vocabulary needed to attack them.

Every row admits more than theorem proving. A mathematician may sharpen a hypothesis or supply a counterexample. A programmer may run a discriminating exact calculation. A formaliser may turn a paper deduction into a checked declaration or find that the two statements disagree. A literature reader may locate a theorem that closes or invalidates a route. The contribution is the exact returned object and its limitation, not the prestige of the problem number.

# 4 The public research object

The unit of participation is a *problem world*. A problem world includes the endpoint question, current public status, principal theorems, formal source, experiments, known counterexamples, failed mechanisms, source literature, and exact open obligations. A bounded task is a neighbourhood in that world, not an invitation to read the repository indiscriminately.

The public layers carry different kinds of evidence. Their order is easier to read as an authority ladder than as one all-purpose badge.

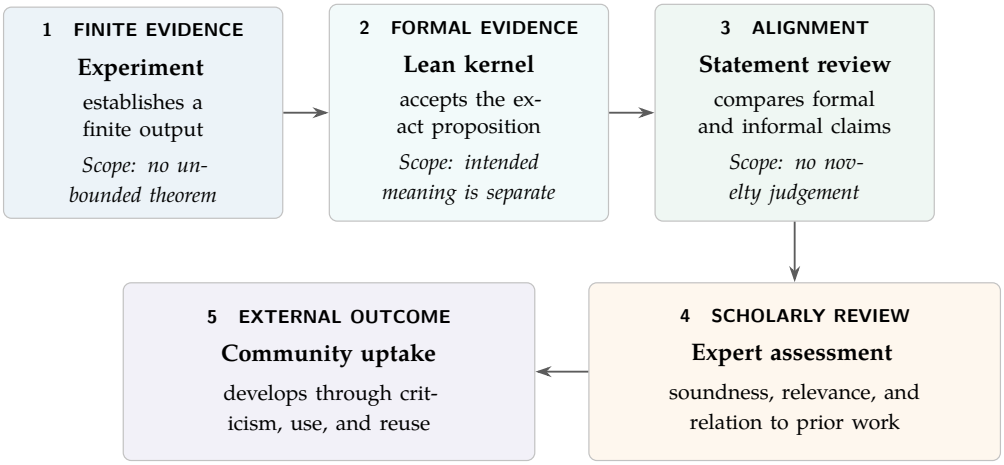


**Figure 4:** Evidence and acceptance remain separate. A candidate need not pass through every box in a single line, but no earlier box inherits the authority of a later one.

The repository uses established tools and contribution practices. Erdős Problems supplies questions, sources, status, and a problem community [11]. Lean supplies the formal language and kernel [12]. DeepMind's `formal-conjectures` supplies a public one-problem-file model with metadata, snapshots, issues, and pull requests [13]. Comparator and Palomar supply exact-interface checking, permitted-axiom inspection, and a durable formal record with a deliberately limited editorial claim [14]. Polymath supplies a social precedent for publishing small, tentative, and negative contributions [7]. BOINC and GIMPS supply the volunteer-compute precedent and make visible why executable work, independent checks, and legible credit matter [5, 6]. The present repository wraps these practices into a route that a newcomer can enter without first rebuilding each component.

The same underlying substrate can support several reader projections: a short public primer, a specialist paper, a detailed proof account, Lean declarations, an agent explanation, and machine-readable queries. Each projection must link back to its source claims and evidence. A shorter or friendlier account does not acquire permission to strengthen them.

An external runner begins at `the compact agent entry`. It can inspect all problem frontiers, choose a bounded question, read the relevant paper and source neighbourhood, run experiments or edit formal code, validate the result, and prepare a return. None of these steps requires access to the private workbench. A contributor is free to replace the agent, the scheduler, or the entire search policy while retaining the public evidence boundary.

# 5 The contribution protocol

A contributor forks the repository, works in a local clone, and opens a pull request with the proposed change. A finding without a patch can be submitted as a research-progress issue; an infrastructure proposal has its own issue form. The `submit-pull-request` skill helps an agent prepare the files, checks, and description. Publishing the branch and request requires the contributor's authorisation.

A return records the starting commit, the result, the evidence, the tools used, and the people who contributed. Proof changes need the appropriate Lean checks. Computations need enough code and input data to reproduce the output. A proposed strengthening must say which hypothesis or conclusion changed. Reviewers then check the result, its relation to the stated problem and prior work, and the changes needed in the paper.

The repository may have moved on while the contribution was being prepared. The starting commit lets a reviewer reconstruct the original change before adapting it to current main. Substantive conflict resolution is recorded as integration work, preserving the original contributor's credit. The final checks run against the version that will be adopted.

Before submission, the contributor also checks what the result affects. A new lemma may change a later proof, a claimed open question, a computation, or a passage in the paper. The `propagate-research-consequences` skill asks for each affected file to be updated, checked as unchanged, or deferred with a reason. After review, an accepted contribution is linked to the commit and files where it was included. Corrections add to that history.

A compute contributor needs a well-chosen task. The default work packet names an open statement, relevant source material, a permitted experiment or proof attempt, and a stopping condition. Preparing useful packets is mathematical work; qualified review should precede large-scale distribution. The public source-packet exporter makes a selected task portable: a contributor chooses committed files and a question, then uploads the resulting text attachment to a model outside the repository harness. The packet preserves the source pin and hashes so that the returned argument can be checked against the same starting material. Selecting enough context remains the contributor's job. That review has not been established for every current task.

Mathematical direction and verification can be contributed separately. Someone may propose a promising question without running an agent, while another contributor examines the eventual result. The record attributes both roles.

Returns are reviewed as mathematics or as changes to the research software. The *mathematics track* accepts proofs, reductions, computations, counterexamples, corrections, formalisation, and reproducible failed routes. The *architecture track* accepts improvements to agent workflow, navigation, validation, reproducibility, public experience, governance, and tooling. An architecture return never needs a fictitious problem number, and it cannot request promotion of a mathematical claim.

The return, review decision, and release remain separate records. A useful correction or design proposal can therefore retain credit even if it does not lead to a merged theorem.

# 6 From an agent claim to mathematical attention

A returned result needs enough preparation that a specialist can assess it. The reviewer should be able to recover the statement, reproduce the evidence, find the relevant prior work, and identify the step that remains uncertain. For a formal proof, this includes checking that the Lean proposition expresses the intended mathematics.

The maintainer first checks whether the return is coherent and relevant to the project. A sufficiently developed result can then be prepared for external attention. Comparator checks selected formal interfaces; Palomar records formal results with structured disclosure and an automated editorial filter. Palomar does not provide human expert review or establish novelty [14]. The Erdős Problems community provides a place to discuss results relevant to its questions, including links to longer arguments [11].

Independent replay, a resolved objection, and a clear explanation give an expert specific reasons to spend time on a result. The project should preserve those contributions alongside the proof, so the next reviewer need not reconstruct the entire discussion.

# 7 Progress, provenance, and credit

Credit records what each contributor did. It can name the author of a theorem, the person who found a counterexample, the contributor who formalised a proof, and those who corrected, reviewed, or explained it. Software and computation receive credit too. The current receipt schema optionally uses the fourteen CRediT roles, which describe contributions without deciding authorship [17].

Credit travels with the repository. After a maintainer accepts a contribution, its record links the people and work to the evidence, public files, and commit where it was included. The public contribution pages are generated from these committed records, so a reader can follow a credit entry back to the work it recognises. Later corrections add to the history. The project does not require ownership of an underlying idea in exchange for including it. Authorship of a later paper or library contribution is a separate decision based on the work performed.

This distinction matters when a local lemma is generalised. The original observation should remain cited, while the mathematician and Lean contributor who develop the general theorem and library interface receive credit for that work. A reference back to the motivating problem need not compete with their authorship.

### Later uses of a contribution

A contribution's importance may become clearer through later work. The record can link an earlier observation to a theorem that uses it, a correction, or a new application. These links are claims to inspect and revise, rather than points on a leaderboard. The original evidence and credit remain available even when no further use has been found. Financial rewards would require a separate policy; the present record promises attribution.

## 8 Distributed compute without distributed authority

Volunteer compute has an established precedent. BOINC packages scientific jobs for heterogeneous consumer devices, and BOINC Central explicitly aims to make volunteer computing available to scientists without the resources to operate their own project [5]. GIMPS shows a mathematical version: volunteers run a common search program, independent machines confirm a prime, and discovery credit includes the compute donor, software authors, server operator, and wider volunteer effort [6].

Agent-assisted research is less uniform than either example. Most tasks are not interchangeable work units with a predetermined verifier. A runner may change code, propose a conjecture, or misread the problem. The public system therefore distributes *search* while keeping authority local to each evidence type. Mathematicians and formalisation contributors design or review the routes; volunteers may execute them with Claude Code, Codex, Cursor, Antigravity, another open or proprietary harness, or no agent at all. The runner is replaceable. The task packet, evidence boundary, and return record are the shared protocol. Returned code is untrusted until reviewed. Expensive or privileged continuous-integration jobs must not execute fork code with repository secrets. In particular, a GitHub `pull_request_target` workflow must not check out and execute an untrusted fork; GitHub documents that combination as a route to secret leakage and repository compromise [18]. Initial checks should run with read-only permissions, no secrets, bounded resources, and explicit artefact retention. Promotion to trusted infrastructure is a later review decision.

A useful compute return records at least the source commit, model and runner, tool disclosure, prompt or task packet, wall-clock time, human time, token or subscription use, and hardware budget where available, together with commands, changed paths, outputs, validation receipts, failures, and the stopping rule. These inputs should not be forced into one exchange rate: a mathematician's hour, an API bill, and a donated graphics processor are different resources. They can still be disclosed well enough to make a capability comparison interpretable and to reveal when hidden scaffolding paid most of the cost.

The first public mode may remain deliberately simple: contributors clone the repository and run their own agents locally. A later volunteer-compute layer could distribute signed, immutable task packets and receive result bundles without granting write access. It should be built only after task identity, sandboxing, deduplication, resource budgets, result replay, and abuse handling have explicit owners.

## 9 Related projects

The contribution process draws on public mathematical collaboration, shared formal libraries, and volunteer computing. The following comparisons explain which practices are being reused.

**Distributed compute.** BOINC and GIMPS show that members of the public will donate hardware to a scientific objective when the client is easy to run, the work is visible, and credit is legible [5, 6]. Their tasks are much more mechanically uniform than open-ended proof research.

**Distributed mathematical insight.** Polymath projects invite participants at different mathematical levels to share small observations as they occur. Their rules explicitly welcome tentative and negative insights, provided they are made clear enough for others to absorb, and treat the project as collaboration rather than a race [7]. Polymath supplies a social protocol; it does not supply a formal proof, computation, and agent-return substrate for every comment.

**Distributed formalisation.** Mathlib uses ordinary fork-and-pull-request practice, human review, and source-level attribution. The Carleson formalisation used a public blueprint and many claimable

lemma-sized tasks, with formalisation feeding corrections back into the informal plan [8, 9]. The blueprint layer itself is `leanblueprint`, which records the Lean declarations that realise a statement, the statement and proof dependency edges an author declares, and a formalisation-readiness state for each node, and which checks that every named declaration exists in the project or one of its dependencies [10]. Statement-to-declaration linking is therefore established practice, and this project claims none of it as new; what it adds is described in the composition below. Google DeepMind's `formal-conjectures` repository similarly uses one-problem files, issue assignment, pull requests, metadata, and stable snapshots for a growing Lean benchmark [13]. These projects demonstrate that large checked mathematical objects can be made publicly divisible.

**Multi-agent formal research.** Agent Hunt studies bounties, locks, guarded ownership, and collaborative agents for autoformalisation [15]. Lean Atlas uses formal dependency information to reduce the declarations a person must inspect for semantic verification [16]. LeanMarathon adds a durable proof blueprint, scoped agents, and checked parallel integration for research-paper formalisation, including prose stored beside formal statements and checked against their proof dependencies [1]. These are close precedents for the working environment. The proposal here is to make its continuing mathematical record and infrastructure jointly contributable, with evidence and credit attached to accepted changes.

**Proof abundance.** Tao separates problem solving into generation, verification, exposition, digestion and acceptance, and canonicalisation, and argues that proof abundance will create bottlenecks between these stages [2]. The strategy here treats those bottlenecks as contribution surfaces. A reviewer who prevents an overclaim or an expositor who makes a hard step recoverable is not ancillary to the research pipeline.

The proposed composition joins these precedents. It combines volunteer compute, small shared insights, formal task decomposition, agent-native problem worlds, explicit negative knowledge, untrusted public returns, role-aware credit, and human review. The maintained object is the relationship among public mathematical claims, the several evidence classes that support them, the failed routes whose obstructions have themselves been proved, the generated public projections, and a research task that continues after publication. The candidate contribution is this composition and its implementation in one open mathematical corpus. The paper reports no controlled comparison showing that the composition increases discovery rate.

# 10 Adding another problem world

The architecture is intended to hold more than the present eight problems, but adding a folder is not enough. A coherent new world needs a canonical question and primary source; a dated account of its current status; an explicit public claim boundary; a readable primer or problem note; formal statements with an informal-faithfulness account where formalisation is appropriate; known literature, computations, counterexamples, and failed routes; exact open contribution points; navigation and validation routes; attribution; and a maintainer or review path.

The public skill for adding a problem separates three states. A proposal may begin as a sourced issue. An incubating formal lane may add checked source under a problem-owned namespace while stating that it is not yet a reviewed public claim. A fully indexed world joins the problem registry, paper corpus, query routes, return schema, cold-start inventory, and relevant external crosswalks. These are distinct transitions because a checked proposition, a published paper, a reviewed claim, and a Comparator selection are different facts.

After any of these transitions, consequence propagation inspects the problem registry, paper corpus, query routes, cold-start inventory, validation roster, return schema, and external crosswalks. A new problem is complete at its stated entry level only when every affected consumer has an explicit disposition; a new directory is not itself a lifecycle receipt.

The last transition is not yet a one-command public operation. The current paper-corpus fan-in has a maintainer-owned stage, several checks enumerate the present roster, and the bounded problem index is already close to its size limit. The immediate infrastructure work is therefore to derive rosters from one public authority, admit an explicit incubating state, split verbose detail out of the bounded index, and make an absent external crosswalk record a normal state rather than an error. Until then, the add-problem skill reports the couplings instead of concealing them behind a scaffold command.

Arbitrary depth is a design target, not a measured scaling result. The useful test is whether a cold agent can retrieve the relevant neighbourhood without reading every paper or rediscovering an existing failure as the number and depth of worlds grow. That test should be repeated whenever the roster or navigation machinery changes.

# 11 Participation and growth

A public contribution process must work for readers with different interests. A mathematician needs the exact question and prior results. A compute donor needs a runnable task and a stopping rule. A reviewer needs the claim and its evidence. An agent builder needs a fixed starting revision against which to compare runs. The README and entry tools provide routes for each.

The next useful test is a complete outside contribution: a newcomer chooses a task, returns reproducible work, receives substantive review, and sees the accepted result and credit in the repository. That would test the proposed participation model more directly than clone counts or publicity.

# 12 Evaluation

The strategy should be evaluated at each conversion boundary. Useful initial measures include:

- time from a cold clone to a correctly scoped first task;
- fraction of returned bundles that can be replayed at the stated commit;
- fraction requiring statement or evidence-class correction;
- time from return to first substantive review and to adoption decision;
- repeated-dead-end rate before and after a no-go enters the corpus;
- reviewer time per accepted result family;
- contribution diversity across mathematics, computation, software, validation, exposition, and review;
- correction lineage completeness and attribution retention; and
- change in useful output at controlled compute when navigation or another infrastructure component changes.

The numerator must remain claim-bounded. One accepted counterexample may be more useful than hundreds of generated lemmas. Generated certificate shards must not be counted as independent discoveries. A theorem accepted by Lean but rejected on intended meaning is not a successful public transition. Likewise, a strong negative result can improve the corpus even though the endpoint problem remains open.

Three questions must be evaluated separately. The first asks whether the runtime at a past revision exposes the exact remaining obligation for a stated target and accepts an authored candidate that avoids the later declaration and module names. The second asks whether the semantic layers present at that revision help a reader find existing mathematics and interpret it correctly. The third asks whether the derived layers as they stood in the past shorten the work of producing a later result. The implemented harness answers the first. It supplies its own authored candidate, names the candidate declaration inside its request, runs only the derived-layer arm, and observes no reader and no agent, so it leaves the second and the third open. Its clone carries the past source with the dependencies of the current tree, so its receipt records the pins that revision declared alongside the pins the build used. Later Git objects, a shared object store, generated dossiers and receipts, network access, context supplied in the prompt, and a model's familiarity with public mathematics can each reveal the target. A comparison that means anything therefore holds the target, the access permissions, and the work budget fixed, and varies only whether the ordinary source-and-documentation route or the derived layers are available. Its outcomes

are stated in advance: correct reconstruction, false strengthening, missed hypotheses, correct identification of the open remainder, successful replay, and usable returned contribution. The channels the harness cannot control are reported as limitations. `docs/methodology.json` carries the channel list and the control status of each one.

Longitudinal model comparisons should use immutable problem snapshots and held-out variants where possible. Public hard problems are susceptible to training-data contamination, and the repository itself will become more informative over time. A later model therefore receives both a stronger model and a richer corpus. The design should measure these factors separately rather than presenting every later success as model improvement.

The present work supplies no such evaluation. It establishes an implemented case, an explicit protocol, and falsifiable measures for a later study.

# 13 What to retain from a research attempt

Retaining a run means selecting the material another researcher could use. A transcript alone is rarely enough. The record should explain the question, the evidence found, and the reason for continuing or abandoning the approach. Search results and computations need source references; a claimed obstruction needs its exact hypotheses.

Review can also change the order of a paper. The agent must compare the new work with the current formal source and literature, including results omitted from the existing manuscript. The strongest useful result should receive the clearest statement and the space needed to explain its hard step. Routine lemmas can remain available in the source without determining the paper's emphasis.

This review starts from the source tree as well as the paper's selected results. A contributor may find an omitted theorem, an argument assembled from existing lemmas, or a useful proof that has not reached the public account. The work then includes checking that argument, formalising a missing step when needed, and updating the paper and its reading routes. Such a contribution should be recorded and credited even when it adds few declarations. Its value is what another researcher can now understand or use.

A problem with the tools is recorded separately. A query that misses a known lemma may justify a retrieval change; a recurring proof-check failure may justify clearer instructions. Before generalising such a lesson, the agent checks whether it recurs in other relevant cases. The repaired instruction then applies to later work. It does not alter the status of a mathematical statement. The public maintenance skill asks the next contributor to replay the repaired journey: find the relevant instructions, run the command, and follow its result. A reusable repair includes that check and the lesson in the owning skill, so it survives beyond the conversation that produced it.

## 13.1 Generalising a useful lemma

A lemma developed for one problem may have a more general use. Finding that use requires further mathematics: identify which hypotheses the proof needs, look for an existing theorem, and formulate a statement that includes the original result as a special case. Proposed weakenings need counterexample checks, and the general proof needs verification. Another application can show whether the generalisation is worth maintaining.

Figure 5 follows the proposed route into a shared library. A mathematician familiar with the subject and a Lean contributor would need to judge the statement, proof, and library interface before opening an upstream discussion.

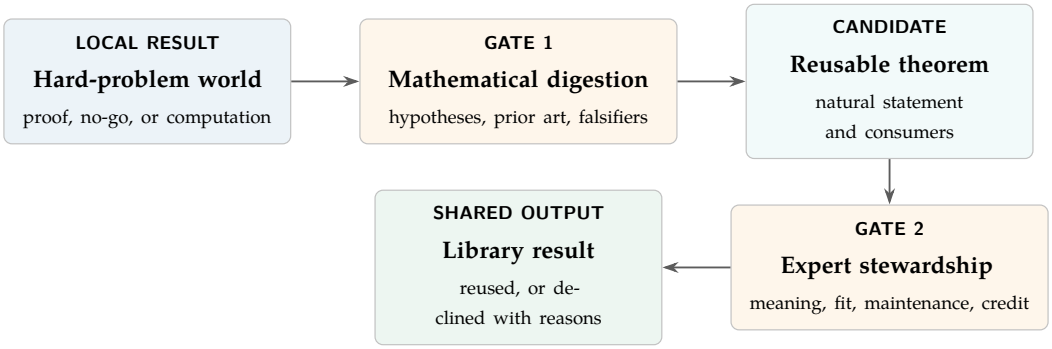


**Figure 5:** The proposed route from a problem-specific lemma to a shared library. The generalisation and library review are additional work.

Mathlib is the clearest external model for the final stage, and also a warning against automating it badly. Its contribution guide welcomes useful contributions and supplies a public fork-and-pull-request route; it does not reserve contribution to people with institutional credentials. It also sets high standards for generality, integration, maintainability, style, and documentation, and its current AI policy rejects low-quality unsupervised agent submissions while requiring AI use to be disclosed [8]. Accordingly, this project should never direct a compute donor to spray raw agent lemmas into Mathlib. A subject and Lean expert must take custody of a candidate, understand it, decide whether it belongs, bring it to community standards, and own the discussion. The public contribution is then theirs in the roles they performed; the originating problem route remains a provenance fact, not a competing claim of ownership.

The present prototype does not automate this track and reports no Mathlib contribution produced by it. It can preserve local evidence and provenance, and it can help prepare a candidate. Canonical status and upstream acceptance remain external outcomes. The systems paper's mathematical loop describes the authority separation underneath this handoff.

The resulting record can connect a local lemma to a later generalisation, an application, or a correction. It can also retain unsuccessful arguments with the reason they fail. These records may be useful for retrieval or future model training. Whether they improve mathematical originality remains an experimental question.

## 14 Possible applications outside mathematics

A later extension could investigate whether the same record-and-review approach helps simulation or analysis of public scientific data. Physical experimentation would additionally require domain expertise, calibrated instruments, uncertainty analysis, controlled facilities, and safety review. The mathematical prototype supplies no evidence of those capabilities; autonomous physical experimentation is not a present project goal.

## 15 Limits and governance

The prototype had no recorded external user or accepted outside contribution by 31 August 2026. Setup and review may therefore have problems that an author-operated rehearsal misses. A clean reproduction or a report of a broken instruction would be useful evidence about the contribution process.

Decisions also remain concentrated in one maintainer. Outside reviewers and maintainers would need a part in problem selection, adoption decisions, and credit disputes. Published correction and appeal procedures would make those decisions open to challenge.

More agents could increase the review burden without producing worthwhile mathematics. Evaluation should therefore account for rejected work, reviewer time, and possible exposure to the public corpus during model training. Contributors have unequal access to compute and expertise; sharing the software addresses only part of that difference.

## 16 Execution order

The immediate task is to test the local contribution process with outside users: finding a question, running the relevant tools, returning evidence, and receiving review and credit. The README, query tools, return schemas, and contribution records supply the starting implementation.

External reviewers and maintainers are needed before a substantial increase in agent throughput. Controlled comparisons can then test changes to retrieval, models, and compute budgets against fixed source revisions. A hosted volunteer scheduler, public task market, or cross-domain service would be later work; none is a reported capability of this release.

## 17 Conclusion

The proposal is to share a usable research environment together with its mathematical results. Each accepted contribution can improve the problem record or the tools used to investigate it, and its credit remains attached. A contributor can therefore advance one part of a difficult problem without reconstructing the entire project.

The public repository implements this contribution path for eight open Erdős problems. Whether it becomes a productive research commons depends on external use: contributors finding worthwhile tasks, reviewers being able to check the returns, and later work benefiting from what was retained. Those are the next outcomes to measure.

## A Public entry routes

The public repository is `wcook04/plectis-erdos`. The shortest current routes are:

| Question | Public route |
|---|---|
| What is the experiment? | `README.md` |
| Where should an agent begin? | `AGENTS.override.md` |
| How can an agent explain the system? | `explain-public-system skill` |
| How can an agent run the coupled research lifecycle? | `coupled-goal skill` |
| How can an agent mine a frontier? | `mine-open-problem skill` |
| How is a bounded Lean change validated? | `lean-concurrent-validation skill` |
| How are downstream consequences reconciled? | `propagate-research-consequences skill` |
| How are clone skills installed elsewhere? | `install-clone-skills skill` |
| What is proved and what remains open? | `docs/RESULTS.md` |
| Which papers and problems exist? | `docs/papers/README.md` |
| How can I contribute mathematics? | `CONTRIBUTING.md` and the research-progress issue form |
| How can I improve the architecture? | `architecture contribution guide` and the architecture-proposal issue form |
| How can an agent prepare a pull request? | `submit-pull-request skill` |
| How can I propose or add another problem? | `add-open-problem skill` |
| How is a return validated? | `erdos-research-return skill` and the `research-commons protocol` |
| How is credit recorded? | `credit policy` |
| What do Comparator and Palomar establish? | `Comparator guide` and `Palomar qualification` |

## Declaration of generative AI use

Every word of this manuscript was generated by agents based on large language models operating within Will Cook's private research system for artificial intelligence. The formal proofs and repository software were likewise drafted and revised by the agents through that system under Cook's direction. Cook set the objectives and acceptance criteria and maintains the research infrastructure. Source-bound mathematical interpretation was performed by the agents, not independently verified by Cook. Cook assumes responsibility for the accuracy, interpretation, and presentation of the work. Generative systems are production tools, not authors, and supply no independent authority.

## References

[1] Y. Zhang, Y. Sun, T. Suzuki, J. D. Lee, and F. Liu, *LeanMarathon: Toward Reliable AI Co-Mathematicians through Long-Horizon Lean Autoformalization*, 2026, arXiv:2606.05400, Sections 2–4.

[2] T. Tao, *Mathematics in the age of AI*, 2026, arXiv:2608.16753.

[3] T. Tao, discussion of learning from studying a mathematical problem, Mastodon thread, 3 September 2026, opening post, accessed 5 September 2026.

[4] T. Tao, discussion of exploration and insight in the Navier–Stokes regularity problem, Mastodon thread, 3 September 2026, opening post; part 5 on preserving exploration, accessed 5 September 2026.

[5] D. P. Anderson, *BOINC: A Platform for Volunteer Computing*, Journal of Grid Computing 18 (2020), 99–122, DOI.

[6] Great Internet Mersenne Prime Search, *GIMPS*, project documentation and discovery-credit record, mersenne.org, accessed August 2026.

[7] Polymath Project, *General polymath rules*, project rules, accessed August 2026.

[8] Lean community, *Contributing to mathlib*, contributor guide, accessed August 2026.

[9] L. Becker et al., *A Blueprint for the Formalization of Carleson's Theorem on Convergence of Fourier Series*, 2025, arXiv:2405.06423.

[10] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, software repository, accessed September 2026.

[11] T. F. Bloom, *Erdős Problems*, problem database, sources, and forum, erdosproblems.com, accessed August 2026.

[12] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, Automated Deduction—CADE 28 (2021), 625–635, DOI.

[13] Google DeepMind, *formal-conjectures*, software repository, accessed August 2026.

[14] Palomar Registry, *About Palomar* and *Contribution policy*, registry documentation and submission standard, accessed August 2026.

[15] C. E. Brown, C. Kaliszyk, and J. Urban, *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*, 2026, arXiv:2603.06737.

[16] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, arXiv:2604.16347.

[17] NISO, *CRediT: Contributor Roles Taxonomy*, role definitions, accessed August 2026.

[18] GitHub, *Preventing pwn requests*, GitHub Actions security guidance, documentation, accessed August 2026.